

# \*\*Comprehensive Guide to Digital Marketing: Strategies, Channels & Analytics\*\*

---

## \*\*1. Digital Marketing Framework & Strategy\*\*

### \*\*1.1 Modern Marketing Funnel\*\*

``yaml

# marketing-funnel.yml

Awareness\_Stage:

Channels:

- SEO
- Content Marketing
- Social Media
- Paid Advertising
- Influencer Marketing

Goals:

- Brand Awareness
- Traffic Generation
- Audience Building

Metrics:

- Impressions
- Reach
- Website Traffic
- Social Engagement

Consideration\_Stage:

Channels:

- Email Marketing
- Retargeting
- Webinars
- Case Studies
- Comparison Content

Goals:

- Lead Generation
- Engagement
- Nurturing

Metrics:

- Lead Conversion Rate
- Email Open Rates
- Content Downloads
- Time on Site

Conversion\_Stage:

Channels:

- Sales Funnels
- Landing Pages
- Chatbots
- Sales Calls
- Demo Requests

Goals:

- Sales
- Sign-ups
- Purchases

Metrics:

- Conversion Rate
- Cost per Acquisition

- Revenue
- ROI

#### Retention\_Stage:

##### Channels:

- Email Nurturing
- Loyalty Programs
- Customer Support
- Community Building
- Upsell/Cross-sell

##### Goals:

- Customer Retention
- Loyalty
- Repeat Business

##### Metrics:

- Customer Lifetime Value
- Churn Rate
- Repeat Purchase Rate
- NPS Score

#### Advocacy\_Stage:

##### Channels:

- Referral Programs
- Testimonials
- User-Generated Content
- Social Sharing

##### Goals:

- Brand Advocacy
- Word-of-Mouth

- Organic Growth

Metrics:

- Referral Rate
- Social Shares
- Review Ratings
- Advocacy Score

```

### \*\*1.2 Digital Marketing Strategy Template\*\*

```python

# marketing\_strategy.py

from datetime import datetime, timedelta

from dataclasses import dataclass

from typing import Dict, List, Optional

import pandas as pd

@dataclass

class MarketingGoal:

name: str

target: float

deadline: datetime

kpis: List[str]

budget: float

@dataclass

class ChannelStrategy:

channel: str

budget\_allocation: float

```
target_audience: str
key_metrics: List[str]
tactics: List[str]
```

```
class DigitalMarketingStrategy:
```

```
    def __init__(self, business_name: str, timeframe: str):
        self.business_name = business_name
        self.timeframe = timeframe
        self.goals = []
        self.channels = []
        self.budget = 0.0
```

```
    def add_goal(self, goal: MarketingGoal):
        self.goals.append(goal)
        self.budget += goal.budget
```

```
    def add_channel(self, channel: ChannelStrategy):
        self.channels.append(channel)
```

```
    def generate_strategy_report(self):
        report = {
            'business': self.business_name,
            'timeframe': self.timeframe,
            'total_budget': self.budget,
            'goals': [goal.__dict__ for goal in self.goals],
            'channel_strategies': [channel.__dict__ for channel in self.channels],
            'roi_projections': self.calculate_roi_projections(),
            'implementation_timeline': self.create_timeline()
        }
```

```
return report
```

```
def calculate_roi_projections(self):
```

```
    projections = {}
```

```
    for goal in self.goals:
```

```
        # Simple ROI calculation (adjust based on industry benchmarks)
```

```
        projected_revenue = goal.budget * 3 # 3:1 ROI assumption
```

```
        projections[goal.name] = {
```

```
            'budget': goal.budget,
```

```
            'projected_revenue': projected_revenue,
```

```
            'projected_roi': (projected_revenue - goal.budget) / goal.budget * 100
```

```
        }
```

```
    return projections
```

```
def create_timeline(self):
```

```
    timeline = {}
```

```
    current_date = datetime.now()
```

```
    for i, channel in enumerate(self.channels):
```

```
        start_date = current_date + timedelta(days=i * 30)
```

```
        timeline[channel.channel] = {
```

```
            'start_date': start_date.strftime('%Y-%m-%d'),
```

```
            'duration_days': 90,
```

```
            'key_milestones': [
```

```
                f"Setup and launch {channel.channel}",
```

```
                f"Optimize {channel.channel} campaigns",
```

```
                f"Analyze {channel.channel} performance"
```

```
            ]
```

```
        }
```

```
return timeline
```

```
# Example Usage
```

```
strategy = DigitalMarketingStrategy("TechStart Inc.", "Q1 2024")
```

```
# Add Goals
```

```
strategy.add_goal(MarketingGoal(  
    name="Increase Website Traffic",  
    target=10000, # monthly visitors  
    deadline=datetime(2024, 3, 31),  
    kpis=["Organic Traffic", "Referral Traffic", "Social Traffic"],  
    budget=5000  
))
```

```
strategy.add_goal(MarketingGoal(  
    name="Generate Qualified Leads",  
    target=200, # leads per month  
    deadline=datetime(2024, 3, 31),  
    kpis=["Lead Conversion Rate", "Cost per Lead", "Lead Quality Score"],  
    budget=8000  
))
```

```
# Add Channel Strategies
```

```
strategy.add_channel(ChannelStrategy(  
    channel="SEO",  
    budget_allocation=0.3,  
    target_audience="Tech professionals searching for solutions",  
    key_metrics=["Organic Rankings", "Backlinks", "Organic Traffic"],  
    tactics=["Technical SEO audit", "Content creation", "Link building"]
```

```
))
```

```
strategy.add_channel(ChannelStrategy(  
    channel="Google Ads",  
    budget_allocation=0.4,  
    target_audience="High-intent searchers",  
    key_metrics=["CTR", "Conversion Rate", "Cost per Conversion"],  
    tactics=["Search campaigns", "Remarketing", "Shopping ads"]  
))
```

```
# Generate Report
```

```
report = strategy.generate_strategy_report()  
print(report)
```

```
'''
```

```
---
```

```
## **2. Search Engine Marketing (SEM) & PPC**
```

```
### **2.1 Google Ads Campaign Structure**
```

```
```python
```

```
# google_ads_manager.py
```

```
from typing import List, Dict
```

```
from dataclasses import dataclass
```

```
from datetime import datetime
```

```
import pandas as pd
```

```
@dataclass
```



```
class Keyword:

    text: str

    match_type: str # 'exact', 'phrase', 'broad'

    cpc_bid: float

    status: str = 'enabled'
```

```
@dataclass
```

```
class AdCopy:

    headline1: str

    headline2: str

    headline3: str

    description1: str

    description2: str

    final_url: str

    path1: str

    path2: str
```

```
@dataclass
```

```
class AdGroup:

    name: str

    keywords: List[Keyword]

    ad_copies: List[AdCopy]

    default_bid: float

    targeting: Dict
```

```
class GoogleAdsCampaign:

    def __init__(self, campaign_name: str, budget: float, campaign_type: str = 'search'):

        self.campaign_name = campaign_name

        self.budget = budget
```

```
self.campaign_type = campaign_type
self.ad_groups = []
self.settings = {
    'network_settings': {
        'target_google_search': True,
        'target_search_network': True,
        'target_content_network': False
    },
    'bidding_strategy': 'manual_cpc',
    'start_date': datetime.now().strftime('%Y%m%d'),
    'end_date': None
}
```

```
def add_ad_group(self, ad_group: AdGroup):
    self.ad_groups.append(ad_group)
```

```
def generate_keyword_report(self):
    report = []
    for ad_group in self.ad_groups:
        for keyword in ad_group.keywords:
            report.append({
                'ad_group': ad_group.name,
                'keyword': keyword.text,
                'match_type': keyword.match_type,
                'bid': keyword.cpc_bid,
                'status': keyword.status
            })
    return pd.DataFrame(report)
```

```

def calculate_budget_allocation(self):
    total_bid_amount = sum(
        sum(kw.cpc_bid for kw in ag.keywords)
        for ag in self.ad_groups
    )

    allocation = {}
    for ad_group in self.ad_groups:
        ad_group_bid_total = sum(kw.cpc_bid for kw in ad_group.keywords)
        allocation[ad_group.name] = {
            'percentage': (ad_group_bid_total / total_bid_amount) * 100,
            'estimated_daily_budget': (ad_group_bid_total / total_bid_amount) * self.budget
        }
    return allocation

```

# Example Campaign Setup

```
campaign = GoogleAdsCampaign("Software Solution Q1", budget=5000)
```

# Ad Group 1 - Core Features

```

core_keywords = [
    Keyword("best project management software", "phrase", 2.50),
    Keyword("project management tool", "exact", 3.00),
    Keyword("team collaboration software", "broad", 2.25)
]

```

```

ad_copy1 = AdCopy(
    headline1="Project Management Software",
    headline2="Streamline Your Team Workflow",
    headline3="Start Free Trial",

```

```
description1="Manage projects efficiently with our all-in-one platform.",
description2="Collaborate, track progress, and meet deadlines easily.",
final_url="https://example.com/free-trial",
path1="project-management",
path2="free-trial"
)
```

```
ad_group1 = AdGroup(
    name="Core-Features",
    keywords=core_keywords,
    ad_copies=[ad_copy1],
    default_bid=2.50,
    targeting={'languages': ['en'], 'locations': ['US', 'CA', 'UK']}
)
```

```
campaign.add_ad_group(ad_group1)
```

```
# Generate Reports
```

```
keyword_report = campaign.generate_keyword_report()
```

```
budget_allocation = campaign.calculate_budget_allocation()
```

```
print("Keyword Report:")
```

```
print(keyword_report)
```

```
print("\nBudget Allocation:")
```

```
print(budget_allocation)
```

```
...
```

```
### **2.2 PPC Bid Management & Optimization**
```

```

```python
# bid_optimizer.py

import numpy as np

from sklearn.linear_model import LinearRegression

from typing import Dict, List

class PPCOptimizer:

    def __init__(self, historical_data: pd.DataFrame):

        self.historical_data = historical_data

        self.model = LinearRegression()

    def analyze_performance(self):

        """Analyze campaign performance and identify optimization opportunities"""

        analysis = {}

        # Calculate key metrics

        self.historical_data['ctr'] = (self.historical_data['clicks'] / self.historical_data['impressions']) * 100

        self.historical_data['conversion_rate'] = (self.historical_data['conversions'] /
self.historical_data['clicks']) * 100

        self.historical_data['cpa'] = self.historical_data['cost'] / self.historical_data['conversions']

        self.historical_data['roas'] = (self.historical_data['conversion_value'] / self.historical_data['cost']) *
100

        # Segment performance

        high_performers = self.historical_data[

            (self.historical_data['conversion_rate'] > self.historical_data['conversion_rate'].median()) &

            (self.historical_data['cpa'] < self.historical_data['cpa'].median())

        ]

```

```
low_performers = self.historical_data[
    (self.historical_data['conversion_rate'] < self.historical_data['conversion_rate'].median()) |
    (self.historical_data['cpa'] > self.historical_data['cpa'].median())
]
```

```
analysis['high_performers'] = high_performers
```

```
analysis['low_performers'] = low_performers
```

```
analysis['recommendations'] = self.generate_recommendations(high_performers, low_performers)
```

```
return analysis
```

```
def generate_recommendations(self, high_performers, low_performers):
```

```
    recommendations = []
```

```
    # Bid adjustment recommendations
```

```
    if len(high_performers) > 0:
```

```
        avg_high_bid = high_performers['bid'].mean()
```

```
        recommendations.append(f"Increase bids for high-performing keywords by 10-20% (current avg: ${avg_high_bid:.2f})")
```

```
    if len(low_performers) > 0:
```

```
        avg_low_bid = low_performers['bid'].mean()
```

```
        recommendations.append(f"Decrease bids for low-performing keywords by 20-30% (current avg: ${avg_low_bid:.2f})")
```

```
    # Keyword recommendations
```

```
    high_volume_low_ctr = low_performers[
```

```
        (low_performers['impressions'] > low_performers['impressions'].median()) &
```

```
        (low_performers['ctr'] < low_performers['ctr'].median())
```

```
]
```

```
if len(high_volume_low_ctr) > 0:
```

```
    recommendations.append(f"Review and improve ad copy for {len(high_volume_low_ctr)}  
keywords with high impressions but low CTR")
```

```
return recommendations
```

```
def predict_optimal_bids(self, target_cpa: float, max_bid: float = 10.0):
```

```
    """Predict optimal bids using machine learning"""
```

```
    # Prepare features
```

```
    X = self.historical_data[['impressions', 'ctr', 'conversion_rate', 'competition']]
```

```
    y = self.historical_data['conversions']
```

```
    # Train model
```

```
    self.model.fit(X, y)
```

```
    # Predict conversions for different bid levels
```

```
    bid_levels = np.linspace(0.5, max_bid, 20)
```

```
    predictions = []
```

```
    for bid in bid_levels:
```

```
        # Simulate features for this bid level
```

```
        features = X.mean().to_dict()
```

```
        features['competition'] = bid # Assuming competition increases with bid
```

```
        pred_conversions = self.model.predict([list(features.values())])[0]
```

```
        estimated_cost = pred_conversions * bid
```

```
        estimated_cpa = estimated_cost / pred_conversions if pred_conversions > 0 else float('inf')
```

```

predictions.append({
    'bid': bid,
    'predicted_conversions': pred_conversions,
    'estimated_cost': estimated_cost,
    'estimated_cpa': estimated_cpa,
    'within_target': estimated_cpa <= target_cpa
})

# Find optimal bid
viable_bids = [p for p in predictions if p['within_target']]
if viable_bids:
    optimal = max(viable_bids, key=lambda x: x['predicted_conversions'])
else:
    optimal = min(predictions, key=lambda x: abs(x['estimated_cpa'] - target_cpa))

return {
    'optimal_bid': optimal['bid'],
    'predictions': predictions,
    'target_cpa': target_cpa
}

# Example Usage
# Sample historical data
data = {
    'keyword': ['kw1', 'kw2', 'kw3', 'kw4', 'kw5'],
    'impressions': [1000, 2000, 1500, 800, 3000],
    'clicks': [50, 120, 75, 30, 180],
    'conversions': [5, 12, 8, 2, 18],

```



```

'cost': [125, 360, 225, 90, 540],
'conversion_value': [500, 1200, 800, 200, 1800],
'bid': [2.50, 3.00, 3.00, 3.00, 1.80],
'competition': [0.7, 0.8, 0.6, 0.9, 0.5]
}

```

```

df = pd.DataFrame(data)
optimizer = PPCOptimizer(df)

```

```

# Analyze performance
analysis = optimizer.analyze_performance()
print("Performance Analysis:")
for rec in analysis['recommendations']:
    print(f"- {rec}")

```

```

# Predict optimal bids
bid_analysis = optimizer.predict_optimal_bids(target_cpa=25.0)
print(f"\nOptimal Bid: ${bid_analysis['optimal_bid']:.2f}")
...

```

```

---
```

```

## **3. Social Media Marketing**

```

```

### **3.1 Multi-Platform Social Media Strategy**

```

```

```python
# social_media_manager.py
from datetime import datetime, timedelta

```

```
from typing import Dict, List
import pandas as pd
```

```
class SocialMediaPlatform:
```

```
    def __init__(self, name: str, content_types: List[str], optimal_times: List[str]):
```

```
        self.name = name
```

```
        self.content_types = content_types
```

```
        self.optimal_times = optimal_times
```

```
        self.posts = []
```

```
    def schedule_post(self, content: str, content_type: str, scheduled_time: datetime):
```

```
        post = {
```

```
            'content': content,
```

```
            'type': content_type,
```

```
            'scheduled_time': scheduled_time,
```

```
            'platform': self.name,
```

```
            'status': 'scheduled'
```

```
        }
```

```
        self.posts.append(post)
```

```
        return post
```

```
class SocialMediaStrategy:
```

```
    def __init__(self):
```

```
        self.platforms = {}
```

```
        self.content_calendar = []
```

```
    def add_platform(self, platform: SocialMediaPlatform):
```

```
        self.platforms[platform.name] = platform
```

```

def create_content_calendar(self, start_date: datetime, weeks: int = 4):
    calendar = []
    current_date = start_date

    # Content themes for the month
    themes = {
        1: "Product Education",
        2: "Customer Stories",
        3: "Industry Insights",
        4: "Company Culture"
    }

    for week in range(weeks):
        week_theme = themes.get((week % 4) + 1, "General")

        for day in range(7):
            post_date = current_date + timedelta(days=day + (week * 7))

            # Skip weekends for some platforms
            if post_date.weekday() < 5: # Monday to Friday
                daily_posts = self.generate_daily_posts(post_date, week_theme)
                calendar.extend(daily_posts)

    self.content_calendar = calendar
    return calendar

def generate_daily_posts(self, post_date: datetime, theme: str):
    posts = []

```

```
# Platform-specific content generation
```

```
platform_strategies = {  
    'LinkedIn': {  
        'types': ['Article', 'Industry Insight', 'Company News'],  
        'times': ['09:00', '12:00', '17:00']  
    },  
    'Twitter': {  
        'types': ['Quick Tip', 'Industry News', 'Engagement Question'],  
        'times': ['08:00', '12:00', '16:00', '19:00']  
    },  
    'Instagram': {  
        'types': ['Visual Story', 'Reel', 'Carousel'],  
        'times': ['11:00', '15:00', '19:00']  
    },  
    'Facebook': {  
        'types': ['Community Post', 'Video', 'Event'],  
        'times': ['09:00', '13:00', '18:00']  
    }  
}
```

```
for platform_name, strategy in platform_strategies.items():  
    if platform_name in self.platforms:  
        for post_time in strategy['times']:  
            # Create post datetime  
            post_datetime = datetime.combine(post_date.date(),  
                                             datetime.strptime(post_time, '%H:%M').time())  
  
            # Generate content based on theme and platform  
            content = self.generate_content(theme, platform_name, strategy['types'])
```

```

        # Schedule post

        platform = self.platforms[platform_name]

        post = platform.schedule_post(
            content=content['text'],
            content_type=content['type'],
            scheduled_time=post_datetime
        )

        posts.append(post)

    return posts

def generate_content(self, theme: str, platform: str, content_types: List[str]):
    """Generate platform-appropriate content"""
    content_templates = {
        'Product Education': {
            'Article': f"Deep dive: How our solution solves {theme} challenges",
            'Quick Tip': f"Pro tip: Maximize your results with this {theme} strategy",
            'Visual Story': f"See how we approach {theme} differently",
            'Community Post': f"What's your biggest challenge with {theme}? Share below! 🙌"
        },
        'Customer Stories': {
            'Article': "Case study: How [Customer] achieved amazing results",
            'Quick Tip': "Learn from our customers' success stories",
            'Visual Story': "Customer success in action 🎥",
            'Community Post': "Which customer story inspired you most? 💬"
        }
    }

```

```
# Select content type
```

```
content_type = np.random.choice(content_types)
```

```
# Get content template
```

```
template = content_templates.get(theme, {}).get(content_type, f"Engaging content about {theme}")
```

```
return {
```

```
    'text': template,
```

```
    'type': content_type,
```

```
    'theme': theme,
```

```
    'platform': platform
```

```
}
```

```
def analyze_performance(self, platform_data: Dict):
```

```
    """Analyze social media performance across platforms"""
```

```
    analysis = {}
```

```
    for platform_name, data in platform_data.items():
```

```
        df = pd.DataFrame(data)
```

```
        # Calculate engagement rate
```

```
        df['engagement_rate'] = (df['engagements'] / df['reach']) * 100
```

```
        # Platform-specific metrics
```

```
        platform_analysis = {
```

```
            'total_posts': len(df),
```

```
            'total_engagement': df['engagements'].sum(),
```

```
            'average_engagement_rate': df['engagement_rate'].mean(),
```

```

        'best_performing_post': df.loc[df['engagements'].idxmax()].to_dict(),
        'optimal_posting_times': self.find_optimal_times(df),
        'content_type_performance': df.groupby('content_type')['engagement_rate'].mean().to_dict()
    }

```

```

    analysis[platform_name] = platform_analysis

```

```

    return analysis

```

```

def find_optimal_times(self, df):

```

```

    """Find best times to post based on historical performance"""

```

```

    df['hour'] = pd.to_datetime(df['post_time']).dt.hour

```

```

    hourly_performance = df.groupby('hour')['engagement_rate'].mean()

```

```

    best_times = hourly_performance.nlargest(3)

```

```

    return best_times.to_dict()

```

```

# Example Usage

```

```

strategy = SocialMediaStrategy()

```

```

# Add platforms

```

```

linkedin = SocialMediaPlatform(

```

```

    name="LinkedIn",

```

```

    content_types=['Article', 'Industry Insight', 'Company News'],

```

```

    optimal_times=['09:00', '12:00', '17:00']

```

```

)

```

```

twitter = SocialMediaPlatform(

```

```

    name="Twitter",

```

```

        content_types=['Quick Tip', 'Industry News', 'Engagement Question'],
        optimal_times=['08:00', '12:00', '16:00', '19:00']
    )

    strategy.add_platform(linkedin)
    strategy.add_platform(twitter)

    # Create content calendar
    calendar = strategy.create_content_calendar(datetime.now(), weeks=2)
    print(f"Scheduled {len(calendar)} posts across {len(strategy.platforms)} platforms")
    ...

```

### \*\*3.2 Social Media Analytics & Reporting\*\*

```

```python
# social_analytics.py
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta

class SocialMediaAnalytics:
    def __init__(self, data: pd.DataFrame):
        self.data = data
        self.setup_visualization()

    def setup_visualization(self):
        plt.style.use('seaborn-v0_8')
        sns.set_palette("husl")

```



```

def comprehensive_report(self, platform: str = None):
    """Generate comprehensive social media performance report"""
    if platform:
        data = self.data[self.data['platform'] == platform]
    else:
        data = self.data

    report = {
        'summary_metrics': self.calculate_summary_metrics(data),
        'engagement_analysis': self.analyze_engagement(data),
        'content_performance': self.analyze_content_performance(data),
        'audience_insights': self.analyze_audience_growth(data),
        'recommendations': self.generate_recommendations(data)
    }

    return report

```

```

def calculate_summary_metrics(self, data):
    """Calculate key performance indicators"""
    return {
        'total_posts': len(data),
        'total_reach': data['reach'].sum(),
        'total_engagements': data['engagements'].sum(),
        'average_engagement_rate': (data['engagements'].sum() / data['reach'].sum()) * 100,
        'total_followers_growth': data['followers_growth'].sum(),
        'average_ctr': data['clicks'].sum() / data['reach'].sum() * 100
    }

```

```

def analyze_engagement(self, data):
    """Deep dive into engagement metrics"""

    data['engagement_rate'] = (data['engagements'] / data['reach']) * 100

    engagement_analysis = {
        'by_platform': data.groupby('platform')['engagement_rate'].mean().to_dict(),
        'by_content_type': data.groupby('content_type')['engagement_rate'].mean().to_dict(),
        'by_time_of_day': self.analyze_engagement_by_time(data),
        'trend_over_time': self.analyze_engagement_trends(data)
    }

    return engagement_analysis

```

```

def analyze_engagement_by_time(self, data):
    """Analyze engagement patterns by time of day"""

    data['hour'] = pd.to_datetime(data['post_time']).dt.hour

    hourly_engagement = data.groupby('hour')['engagement_rate'].mean()

    return {
        'best_hours': hourly_engagement.nlargest(3).to_dict(),
        'worst_hours': hourly_engagement.nsmallest(3).to_dict(),
        'hourly_pattern': hourly_engagement.to_dict()
    }

```

```

def analyze_content_performance(self, data):
    """Analyze which content types perform best"""

    content_metrics = data.groupby('content_type').agg({
        'engagement_rate': 'mean',
        'reach': 'mean',

```

```

        'clicks': 'mean',
        'engagements': 'mean'
    }).round(2)

    return content_metrics.to_dict()

def analyze_audience_growth(self, data):
    """Analyze audience growth and demographics"""
    growth_analysis = {
        'total_followers': data['followers'].iloc[-1] if len(data) > 0 else 0,
        'growth_rate': ((data['followers'].iloc[-1] - data['followers'].iloc[0]) / data['followers'].iloc[0]) *
100 if len(data) > 1 else 0,
        'platform_breakdown': data.groupby('platform')['followers'].last().to_dict()
    }

    return growth_analysis

def generate_recommendations(self, data):
    """Generate data-driven recommendations"""
    recommendations = []

    # Engagement-based recommendations
    avg_engagement = data['engagement_rate'].mean()
    high_engagement = data[data['engagement_rate'] > avg_engagement * 1.2]

    if len(high_engagement) > 0:
        best_content_types = high_engagement['content_type'].value_counts().head(3)
        recommendations.append(

```

```
        f"Focus on {', '.join(best_content_types.index)} content types which perform 20%+ above  
average"
```

```
)
```

```
# Time-based recommendations
```

```
data['hour'] = pd.to_datetime(data['post_time']).dt.hour
```

```
best_hours = data.groupby('hour')['engagement_rate'].mean().nlargest(3)
```

```
if len(best_hours) > 0:
```

```
    recommendations.append(
```

```
        f"Schedule more posts during peak engagement hours: {', '.join(map(str, best_hours.index))}h"
```

```
)
```

```
# Platform-specific recommendations
```

```
platform_performance = data.groupby('platform')['engagement_rate'].mean()
```

```
best_platform = platform_performance.idxmax()
```

```
recommendations.append(
```

```
    f"Allocate more resources to {best_platform} which shows highest engagement rates"
```

```
)
```

```
return recommendations
```

```
def create_visualizations(self, platform: str = None):
```

```
    """Create comprehensive visualizations"""
```

```
    if platform:
```

```
        data = self.data[self.data['platform'] == platform]
```

```
    else:
```

```
        data = self.data
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
```

```
# Plot 1: Engagement by Content Type
```

```
content_engagement = data.groupby('content_type')['engagement_rate'].mean()
```

```
axes[0, 0].bar(content_engagement.index, content_engagement.values)
```

```
axes[0, 0].set_title('Average Engagement Rate by Content Type')
```

```
axes[0, 0].tick_params(axis='x', rotation=45)
```

```
# Plot 2: Engagement by Hour
```

```
data['hour'] = pd.to_datetime(data['post_time']).dt.hour
```

```
hourly_engagement = data.groupby('hour')['engagement_rate'].mean()
```

```
axes[0, 1].plot(hourly_engagement.index, hourly_engagement.values, marker='o')
```

```
axes[0, 1].set_title('Engagement Rate by Hour of Day')
```

```
axes[0, 1].set_xlabel('Hour of Day')
```

```
axes[0, 1].set_ylabel('Engagement Rate (%)')
```

```
# Plot 3: Platform Comparison
```

```
platform_engagement = data.groupby('platform')['engagement_rate'].mean()
```

```
axes[1, 0].bar(platform_engagement.index, platform_engagement.values)
```

```
axes[1, 0].set_title('Engagement Rate by Platform')
```

```
# Plot 4: Follower Growth Over Time
```

```
if 'date' in data.columns:
```

```
    data['date'] = pd.to_datetime(data['date'])
```

```
    daily_followers = data.groupby('date')['followers'].max()
```

```
    axes[1, 1].plot(daily_followers.index, daily_followers.values)
```

```
    axes[1, 1].set_title('Follower Growth Over Time')
```

```
    axes[1, 1].tick_params(axis='x', rotation=45)
```

```
plt.tight_layout()
```

```
return fig
```

```
# Example Usage
```

```
# Sample social media data
```

```
sample_data = {
```

```
    'platform': ['LinkedIn', 'Twitter', 'Instagram', 'LinkedIn', 'Twitter', 'Instagram'],
```

```
    'content_type': ['Article', 'Thread', 'Reel', 'Industry Insight', 'Quick Tip', 'Carousel'],
```

```
    'post_time': [
```

```
        '2024-01-15 09:00:00', '2024-01-15 12:00:00', '2024-01-15 15:00:00',
```

```
        '2024-01-16 09:00:00', '2024-01-16 12:00:00', '2024-01-16 15:00:00'
```

```
    ],
```

```
    'reach': [1500, 3200, 5800, 1800, 3500, 6200],
```

```
    'engagements': [45, 128, 406, 54, 140, 434],
```

```
    'clicks': [23, 45, 89, 28, 52, 95],
```

```
    'followers': [12500, 8900, 21500, 12550, 8950, 21600],
```

```
    'followers_growth': [50, 50, 100, 50, 50, 100]
```

```
}
```

```
df = pd.DataFrame(sample_data)
```

```
analytics = SocialMediaAnalytics(df)
```

```
# Generate report
```

```
report = analytics.comprehensive_report()
```

```
print("Social Media Analytics Report:")
```

```
print("=" * 50)
```

```
for section, content in report.items():
```

```

print(f"\n{section.upper()}:")
if isinstance(content, dict):
    for key, value in content.items():
        print(f"  {key}: {value}")
else:
    for item in content:
        print(f"  - {item}")

# Create visualizations
fig = analytics.create_visualizations()
plt.show()
...

---
```

**## \*\*4. Email Marketing & Automation\*\***

**### \*\*4.1 Email Campaign Strategy\*\***

```

```python
# email_marketing.py

from datetime import datetime, timedelta
from typing import List, Dict
import pandas as pd

class EmailCampaign:
    def __init__(self, campaign_name: str, audience: str, goal: str):
        self.campaign_name = campaign_name
        self.audience = audience
```

```
self.goal = goal  
self.emails = []  
self.schedule = []  
self.metrics = {}
```

```
def add_email(self, subject: str, content: str, days_after_previous: int = 0):
```

```
    email = {  
        'subject': subject,  
        'content': content,  
        'delay_days': days_after_previous,  
        'type': 'broadcast'  
    }  
    self.emails.append(email)  
    return email
```

```
def add_automation_flow(self, trigger: str, emails: List[Dict]):
```

```
    automation = {  
        'trigger': trigger,  
        'emails': emails,  
        'type': 'automation'  
    }  
    self.emails.append(automation)  
    return automation
```

```
def generate_schedule(self, start_date: datetime):
```

```
    schedule = []  
    current_date = start_date
```

```
    for email in self.emails:
```



```

if email['type'] == 'broadcast':

    schedule.append({

        'type': 'broadcast',

        'subject': email['subject'],

        'scheduled_date': current_date,

        'audience': self.audience

    })

    current_date += timedelta(days=email['delay_days'])

else: # automation

    for auto_email in email['emails']:

        schedule.append({

            'type': 'automation',

            'trigger': email['trigger'],

            'subject': auto_email['subject'],

            'delay_hours': auto_email.get('delay_hours', 0),

            'audience': f"Users who {email['trigger']}"

        })


self.schedule = schedule

return schedule

```

```

class EmailAnalytics:

    def __init__(self, campaign_data: pd.DataFrame):

        self.campaign_data = campaign_data


    def calculate_campaign_metrics(self):

        """Calculate comprehensive email campaign metrics"""

        metrics = {}

```

```
# Basic metrics
```

```
metrics['total_sent'] = self.campaign_data['sent'].sum()
```

```
metrics['total_opens'] = self.campaign_data['opens'].sum()
```

```
metrics['total_clicks'] = self.campaign_data['clicks'].sum()
```

```
metrics['total_conversions'] = self.campaign_data['conversions'].sum()
```

```
# Rate calculations
```

```
metrics['open_rate'] = (metrics['total_opens'] / metrics['total_sent']) * 100
```

```
metrics['click_rate'] = (metrics['total_clicks'] / metrics['total_opens']) * 100
```

```
metrics['conversion_rate'] = (metrics['total_conversions'] / metrics['total_clicks']) * 100
```

```
metrics['overall_conversion_rate'] = (metrics['total_conversions'] / metrics['total_sent']) * 100
```

```
# Engagement metrics
```

```
metrics['ctr'] = (metrics['total_clicks'] / metrics['total_sent']) * 100
```

```
return metrics
```

```
def analyze_subject_line_performance(self):
```

```
    """Analyze subject line effectiveness"""
```

```
    subject_analysis = self.campaign_data.groupby('subject_line').agg({
```

```
        'open_rate': 'mean',
```

```
        'click_rate': 'mean',
```

```
        'conversion_rate': 'mean'
```

```
    }).round(2)
```

```
    return subject_analysis.sort_values('open_rate', ascending=False)
```

```
def segment_audience_performance(self):
```

```
    """Analyze performance by audience segments"""
```

```

if 'audience_segment' in self.campaign_data.columns:

    segment_analysis = self.campaign_data.groupby('audience_segment').agg({

        'open_rate': 'mean',

        'click_rate': 'mean',

        'conversion_rate': 'mean',

        'unsubscribe_rate': 'mean'

    }).round(2)

```

```

    return segment_analysis.sort_values('conversion_rate', ascending=False)

return None

```

```

def predict_optimal_send_times(self):

```

```

    """Predict best times to send emails based on historical data"""

```

```

    if 'send_time' in self.campaign_data.columns:

```

```

        self.campaign_data['send_hour'] = pd.to_datetime(self.campaign_data['send_time']).dt.hour

```

```

        hourly_performance = self.campaign_data.groupby('send_hour').agg({

            ,

```